

Universidad de los Andes
Engineering Faculty
Systems and Computing Department

**Asset Tokenization in Real Estate
through the Means of Token Standards
Available on the Ethereum Blockchain**

Pedro Vallejo Seade
p.vallejo@uniandes.edu.co
201625720

Advisor: Darío Ernesto Correal

December 2022

Table of Contents

<u>ABSTRACT</u>	<u>4</u>
<u>ACKNOWLEDGEMENTS</u>	<u>5</u>
<u>CONTEXT</u>	<u>5</u>
BLOCKCHAIN	5
APPLIED BLOCKCHAINS AND THE ETHEREUM BLOCKCHAIN	7
SMART CONTRACT	8
TOKENS	8
TOKEN STANDARD	9
ASSET TOKENIZATION	10
<u>RESEARCH AND MOTIVATION</u>	<u>10</u>
ASSET TOKENIZATION RESEARCH	10
REAL ESTATE ASSET TOKENIZATION RESEARCH	12
PROJECT MOTIVATION	14
REAL ESTATE ASSET TOKENIZATION POSSIBLE PATH UNDER COLOMBIAN LAW	15
<u>DESIGN</u>	<u>17</u>
CHALLENGES	17
PROJECT SCOPE AND TARGET	18
FUNCTIONAL REQUIREMENTS	19
APPLICATION'S DESIGN AND FLOW	20
<u>TOOLS</u>	<u>21</u>
<u>IMPLEMENTATION</u>	<u>23</u>
SERVER APPLICATION IMPLEMENTATION	24
CLIENT APPLICATION IMPLEMENTATION	33
<u>RESULTS</u>	<u>36</u>
<u>FURTHER WORK</u>	<u>38</u>
FURTHER WORK ON THE SERVER APPLICATION	38
FURTHER WORK ON THE CLIENT APPLICATION	38
FUTURE PLATFORM	38

CONCLUSIONS	39
--------------------	-----------

REFERENCES	39
-------------------	-----------

Abstract

Most of the global wealth at the moment is currently trapped within non-liquid assets which posts an ever-greater barrier on the fast-paced world that exists on this digital era where trillions of transactions are being made each second. The emerging technology of blockchain has proven to have the potential to unlock the liquidity in a wide variety of assets, via a process called asset tokenization. This process is now a quickly growing industry as projections mark that the majority of assets will be tokenized within 10 years.

Asset tokenization is the process of creating a token in a blockchain that represents the ownership of an asset. Digitalizing the representation of ownership of assets has a variety of aspects on the surface that indicate how and why asset tokenization helps with liquidity, one of these aspects being faster transaction times. Another property of tokenization that increases liquidity of an asset is the possibility of fractionization (breaking ownership into smaller pieces). The tokenization of assets before blockchain marked a variety of security and transparency problems that are solved by this revolutionary technology, and for that reason, before asset tokenization was not feasible. But now various types of assets can be safely tokenized, one of these being real estate assets.

Real estate is notoriously an industry with low levels of liquidity, but now this is changing due to asset tokenization and lots of businesses in this market are emerging using asset tokenization as the basis of their business models. Most of these innovations have not seen the same momentum in developing countries as they have in developed countries due to a variety of reasons and barriers to entry.

This project looks to mark a starting point to enable easy access to this ever-growing market to the population with a basic understanding of technology, focusing on Latin American countries and punctually on Colombia.

Acknowledgements

This project, being the conclusion of my bachelor's degree, I can't help but begin my acknowledgements thanking my family and friends, always having their support in more ways than one has been principal reason I have achieved what I have achieved, so I feel incredibly grateful and indebted to all of them and so they need to be the first mentions in this concluding document. Following them, I also feel the need of thanking the professors and teacher assistants that taught me, helped me and in sometimes even inspire me. They have been the ones that laid the foundation which has allowed me to solve the problems that arrived during the development of this project.

Now I must thank the people that had a direct contribution in the project. The first one of course must be my advisor Darío Ernesto Correal. When I approached him only having an interest on the topic of blockchain, he was the one who motivated me to peruse this project, and then during the process of elaboration, he created a framework to follow that allowed me to continually advance and eventually conclude the project. Following him, I must also thank the technology innovation team of Bancolombia, especially Sebastian Ortega, Jeisson and Rafael, their feedback and ideas represented a very important contribution on my project. And finally, I must thank two of my friends, Martin Tellez and Andres Sabogal who were the ones who helped me to elaborate the legal component presented here.

Context

Blockchain

A blockchain in its essence is a distributed ledger, therefore, in order to understand this technology, defining and understanding what a distributed ledger is primordial. "A distributed ledger is essentially an asset database that can be shared across a network of multiple sites, geographies or institutions" (Hancock & Vaizey, 2016). In this environment, every participant will have a copy of the ledger and any events, transactions or changes within the ledger will be

reflected in all the copies in a short amount of time. The way in which these changes are handled will depend on the implementation of each distributed ledger, however in terms of security all these types of systems are maintained cryptographically with keys and signatures that “control who can do what within the shared ledger” (Hancock & Vaizey, 2016).

A blockchain is an implementation of a distributed ledger through a P2P network where every peer represents a participant and will therefore have a copy of the ledger which is the blockchain. Now, the implementation of the actual ledger in this system comes from its very name, it is a chain of blocks. Each of the blocks will contain essentially three things: data, a hash of its content and the hash of the previous block, with the exception of the genesis block. The use of the hashes makes it very easy to identify when a block has been tempered with, as it will not match the hash registered on its following block, and consequently, this will have the effect of making the entire blockchain invalid. However, computers nowadays are able to calculate hashes incredibly quickly, therefore tempering with the whole chain, including the hashes in order to make the blockchain valid again is not difficult, for this reason, blockchains have implemented a mechanism that mitigates this problem called proof-of-work which slows down the creation and changes on its blocks. In simple words, this mechanism is a “cryptographic proof” (Lachtar, Elkhail, Bacha, & Malik, 2020) in which one party is able to insure the use of a specific amount of computational effort to another party. This makes it very difficult to temper with the blocks because apart from calculating all the correct hashes, there needs to be a calculation of all the proofs of work for each block and this is very slow, giving enough time for a system to identify when a blockchain is not valid. In Bitcoin a proof-of-work for one block takes around 10 minutes (Houben & Snyers, 2018).

Wrapping all of these ideas, blockchain in a nutshell is a decentralized network that provides a series of complexities to ensure that the information within the network is reliable, transparent and therefore can be trusted. Although this technology involves a variety of

interesting mathematics and technologies, for the purpose of this project, this understanding of what is a blockchain is enough.

Applied Blockchains and The Ethereum Blockchain

The first widely used blockchain network was the blockchain of Bitcoin, released as an open-source code by an unknown founder that goes by the name of Satoshi Nakamoto (Davis, 2011). Bitcoin was developed as a means to “solve the double-spend problem¹” when the founder identified that a blockchain could potentially solve this problem and also “sought to disintermediate the central banking powers” (Tapscott & Tapscott, 2016). Sometime after its release the bitcoin blockchain became the most widely used blockchain and the main cryptocurrency on the market. This blockchain keeps growing, becoming more robust and trustworthy through miners, who are the people that give the computational power to do the proof-of-work, these people are then rewarded by transaction fees from the transactions they included in the block and also with newly created bitcoins (Vance, 2013).

Borrowing some ideas from Bitcoin, Ethereum was “conceived in 2013” as a new blockchain that took around 18 months to develop by a nineteen-year-old Russian Canadian hacker Vitalik Buterin, the Ethereum blockchain was then launched on July 30, 2015 (Tapscott & Tapscott, 2016). This new blockchain came into fruition after the founder wanted to resolve some problems he saw on the Bitcoin blockchain, but also wanted to extend the reach of this the network by not only being host to cryptocurrencies but also “run decentralized application, namely smart contracts exactly as programmed without any possibility of downtime, censorship, fraud or third party interference.” (Tapscott & Tapscott, 2016)

¹ The double spend problem is an issue identified on digital cash where “the same digital token could be spent more than once” (Usman, 2021)

Smart Contract

The term smart contract was first coined in 1995 by Nick Szabo who describe them as:

A computerized transaction protocol that executes the terms of the contract. The general objective of smart contract design is to satisfy common contractual conditions (such as payment terms, liens, confidentiality, and even enforcement), minimize exceptions both malicious and accidental, and minimize the need for trusted intermediaries (Szabo, 1994)

But a more comprehensible understanding for what a smart contract is within the blockchain is a code with that “is deployed using cryptographically signed transaction on the blockchain network” (Yaga, Mell, Roby, & Scarfone, 2018). These smart contracts are then executed by the nodes of the blockchain network, and the result of the execution is then registered on the blockchain.

The code for smart contract typically represents a multi-party transaction that represents a business process and the benefits that come from it are “attestable data and transparency that can foster trust” compared to the traditional business applications and processes. (Yaga, Mell, Roby, & Scarfone, 2018)

Tokens

Tokens are “units of value” within a blockchain organization (Cryptopedia Staff, 2022). Therefore, they serve as the main form of asset within a blockchain network, and there are two main types of tokens: fungible and non-fungible tokens.

Fungibility in economics is a term used to describe the assets which are in terms of value equal, interchangeable substitutable and uniform (Market Business News, 2021). For example, any one-dollar bill is interchangeable with any other dollar bill, however if this bill is somehow signed or gained any particularity that made it more or less valuable than one dollar, the bill would lose its fungibility (Osorio, 2022). In simple terms tokens can be

understanded as a way to describe any form of asset, whether is something like a currency where all the tokens are interchangeable and have the same value (fungible tokens), or something that has its unique value, and it is not fundamentally indistinguishable or interchangeable with another asset such as a house.

Token Standard

A token standard is a set of smart contracts that follow the same rules and have the same functionalities. Before a token standard is officially used within the Ethereum Network it must pass a set of stages. The first stage is becoming an EIP (Ethereum Improvement Proposal) which in its essence is a design document that proposes some changes in the Ethereum network. Once this design has passed through different stages and iterations of acceptance, it can become an ERC (Ethereum Request for Comment) and then be used as an actual token within the Ethereum network (Osorio, 2022).

In the Ethereum network alone there are already hundreds of token standards with thousands of EIPs awaiting to become standards, but the main ones in use to this date are the following:

- ERC-20: The most popular token especially when it comes to fungible tokens.
- ERC-721: The main standard for non-fungible tokens.
- ERC-777: An alternative for fungible tokens that gets rid of the double transaction verification of ERC 20, lowers the transaction overhead, and allows users to reject incoming tokens.
- ERC-1155: A non-fungible token with the possibility of having fungible token qualities and has some other improvements as lower transaction cost among others.

Asset Tokenization

Asset tokenization is the process of representing the ownership of any type of asset within a blockchain, and it is foreseen to become the principal ownership medium in the digital age. Tokens are representing the ownership interests and becoming an infrastructure of its own. In order to comply with the law's tokens have even extended a branch called STOs, which basically are tokens that within their set of rules and data are constructed in such a way that it will comply with a set of laws.

Asset tokenization [...] has shown great potential to overcome current constraints on public and private finance. Tokenization enables the digital representation of assets on distributed ledgers or the issuance of securities in the tokenized form. The process allows the conversion of the economic value and ownership rights derived from underlying assets in the off-chain world into digital tokens on the blockchain. (Tian, et al., 2021)

Research and Motivation

Asset Tokenization Research

Most of the wealth available on the world today is illiquid² and typically the “trade discount [from illiquid assets] vs liquid assets are characterized by a high stock-to-flow ratio, lowering the trading volumes and imperfect price discovery” (Kumar, Suresh, Liu, Kronfellner, & Kaul, 2022). These are the principal reasons for asset illiquidity:

1. Limited affordability of mass investors
2. Inability to fractionalize
3. Lack of information
4. Limited access
5. Regulatory Hurdles

² In economics and finance liquidity refers to the “trade-off of the price that an asset can be sold and how quickly it can be sold” (Keynes, 1930)

6. Complex user's journey for having access

(Kumar, Suresh, Liu, Kronfellner, & Kaul, 2022)

Most of these problems can be resolved with asset tokenization. Asset tokenization allows fractionalization of practically any asset like land, real state and even art assets are now being fractionalized with the use of tokenization increasing their overall liquidity (Yip, 2022). Asset tokenization also solves the problems of limited access and complex user's journey for access as now the ownership of these assets is available in some cases one click away through the blockchain. Regulatory hurdles are the last of the illiquidity problems mentioned above that tokenization has the potential for solving with STOs (Security Token Offering), however in this case is very dependable of each of the cases how much of this illiquidity problem can be solved.

Due to this capacity of solving the illiquidity problems that are holding most of the wealth in the world, the market for asset tokenization is foreseen to boom in the next years. A Boston Consulting Group report published on 2022 studied this new market and its potential. The results seen currently are that in 2021 this market reached a value of 2,300 million dollars which marked a 500% increase in comparison to 2020 (Kumar, Suresh, Liu, Kronfellner, & Kaul, 2022). The report also followed to project that the market will reach a total value of 16.1 billion dollars by 2030, and this is stated in a remark as a "conservative figure". This is an incredible potential for a market; however, this report also mentioned some limitations which will be discussed afterwards.

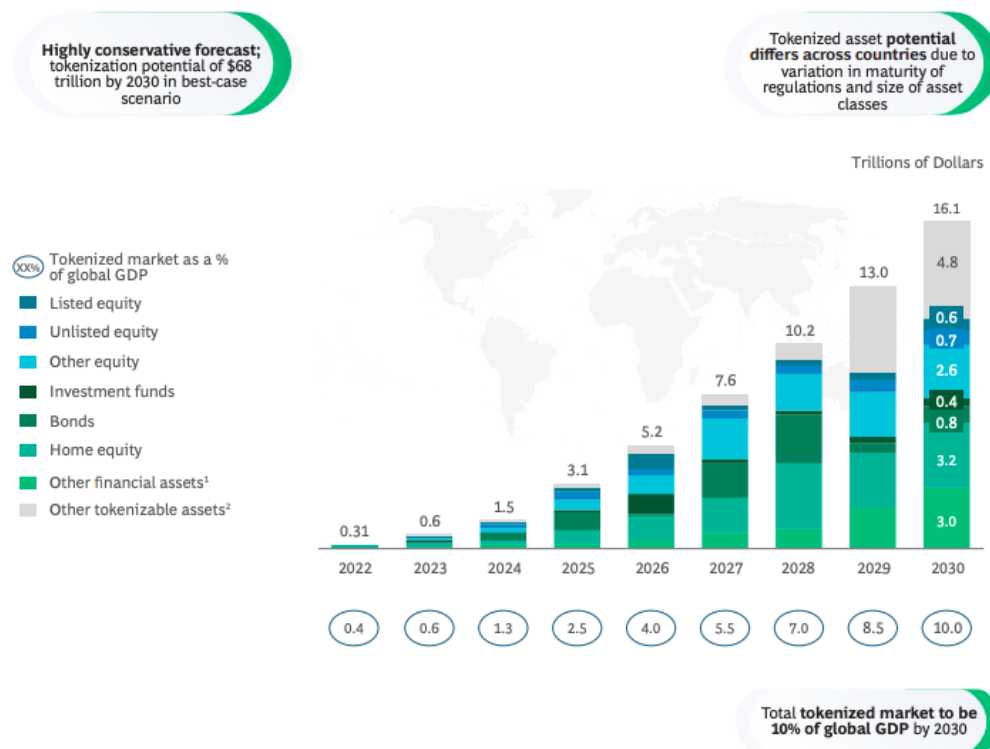


Figure 1: Tokenization of global illiquid assets
 [Source World Economic Forum -Global Agenda Council, BCG Analysys]
 (Kumar, Suresh, Liu, Kronfellner, & Kaul, 2022)

Real Estate Asset Tokenization Research

Real estate in itself is a complex asset class that “makes up a significant economic global segment in terms of the asset base and the transactional activity” (Gupta, et al., 2020). There have been some advances in the field, however, none of them very significant to really alter the market. Asset tokenization has this potential with its capacity of increasing liquidity and nowadays it is even opening the market to retail investors. There is a wide use of fungible tokens such as an ERC-777 representing the fractional ownership of the real estate that can be purchased by investors and be listed on secondary exchanges.

The current state of real estate market has a variety of drawbacks when it comes to investment markets most of which are associated with the liquidity of the asset. Real Estate Investment Trusts (REIT) was a movement that helped the market to mitigate some of its illiquidity problems, but international markets are unreachable and there are high management and transaction fees. Thus, this revolution on this market did not mark a big change.

Asset tokenization has already proven to increase liquidity and investors are being very attracted to the transparency, accessibility, and the faster and cheaper transactions. Due to this, new businesses have pioneered their way into the real estate tokenization market, the following are a few of them:

- Reental: A Spanish company that allows real estate investments from 100 euros. It has around 7,200 users and have tokenized 25 properties using fractionalization
- Propy: A platform that allows the clients to buy and sell properties using cryptocurrencies, NFTs and also traditional payment methods. This platform acts as a market of real estate transactions that has the mission of reducing the entry barriers to the market. The developers at propy through smart contracts automatize the transactions of all the assets within the platform
- Estate Protocol: A platform based on the Ethereum blockchain that presents an NFT market backed with real estate assets and facilitates the investment and fractionalization of all its assets.
- Domoblock: Another investment company where the clients can invest on tokenized assets from 200 euros.
- RealT: Another platform that offers simplified investment in real estate by having the ownership of a property be represented by a digital token in the Ethereum blockchain.

(Maldonado, 2022)

This are just some of the many companies and businesses that are pioneering into this new market. Upon researching, it is noticeable an expected trend where most of these

businesses are either in North America, Europe, and some in Asia. On South America however, this market has not been explored in a noticeable amount.

Another aspect of these companies is the fact that they are working with STOs, which make sense as they all are tokenizing real estate and this real estate inevitably will be tied to the laws of ownership of each country where the asset resides. However, none of them, apart from RealT disclose how they are creating these tokens that comply with the federal law of the place where their property resides.

Project Motivation

The research summarized opened the path for the motivation of this project, as looking through this, there was a clear necessity that is implied. The first two parts of the necessity comes from the report by the Boston Consulting Group, where most noticeably expresses an insurmountable increase in the market and therefore a big space to enter within it. On the other hand, apart from looking at the current and potential market of asset tokenization, it also listed a variety of obstacles faced by this market that impedes its potential growth. The following are the obstacles listed:

- Variance in the regulations across key markets
- Unclear protocols on managing the tussle between TradFis
- Awareness
- Adoption
- Technical barrier of entry

The latter one represents the second necessity for this project. In order to tokenize an asset, it is required a set of technical abilities that are not widely available to most people, especially in developing countries, therefore its reachability becomes very small.

The last necessity comes from the research of current businesses that are exploring this market and the lack of this innovation within Latin America, partly connected to the previously mentioned necessity that implies the lack of technical abilities in developing countries.

Gathering up all these necessities comes the basis of this project. To be a first step to facilitate the access of asset tokenization to any user with enough technical abilities to manage a wallet where the tokens will be stored and give a possible path for Latin American countries to enter this market

Real Estate Asset Tokenization Possible Path Under Colombian Law

This section of the project will explore very briefly and simply a possible path to follow in order to connect the asset tokenization with Colombian laws on property ownership. Hopefully this brief section can be of use for a deeper and more comprehensible investigation to provide a path for asset tokenization that complies with Colombian or another Latin American country's law. The importance of this brief section is the awareness that technical projects will continue to be theoretical when they are not aware of the context where they are implemented.

Two recently graduated Colombian lawyers were consulted for this section of the project. The first step in order to create this path was to see what was already in use and available and as it was mentioned before, none of the companies that are creating this STOs are disclosing the technicalities behind, with the exception of RealT. RealT in their web page on the frequently asked question section disclose how their tokens are compliant with United States federal law. They express that there is no possible way to represent ownership of a property through tokenization at the moment, however there is a path to tokenize the interest of a Limited Liability Company (LLC) that is the sole owner of the asset. Therefore, by having possessed a token from RealT indirectly makes the user the owner of the Real Estate Asset. (RealT, 2021)

This alternative however was not useful as there was no way to make a token represent ownership either of a property or a company that owns said property in Colombia. Not finding any other explicit way that tokenization of assets is being made legal, it was needed to begin therefore from the legal basis of ownership of properties within Colombia in terms of law, which are in its essence four steps:

1. There is an agreement between both parties (the owner and the acquirer of a property)
2. The lawyers of both parties create a document where the transaction is formalized
3. The latter document is then passed to a government office of the territory where the property resides called *oficina de registros de instrumentos públicos*.
4. This office then registers on a document called *certificado de tradición y libertad*, that essentially is a document that registers the history of the property activity such sales, mortgages, and other relevant legal history. If everything is correct, the new ownership of the property is formalized.

By inspecting this, on the surface there is a great potential for the digitalization of this process with the use of blockchain. At the moment asset tokenization and blockchain could feasibly be used to help with the first two steps of the process. However, in the future, it has the potential to be used on the entire process as blockchain can serve as this public office and a token represent the official document of legal history as all the transactions of a token stay registered within the blockchain, and this process is transparent, reliable and will be faster than the current process.

The previous is just a vision for the future, but the countries laws must evolve in order to allow this digitalization of the process, and potentially make the ownership of a token directly related to the ownership of a property. Nevertheless, as it was previously stated, asset

tokenization at the current moment may only take the first two steps of the process. This is possible through the means of a fiduciary contract where the current owner of the property gives it to a fiduciary where the property gains a legal status of being owned by no one, formally called an autonomous patrimony, until the terms of the fiduciary contract are met or broken. These terms of the contract can be associated with the effective transaction of a token through the means of a smart contract, if the transaction was completed, ownership can then be passed to the new party, and if not, ownership will be restored to the previous owner, this process is resumed on figure 2 (Gerencie, 2021).

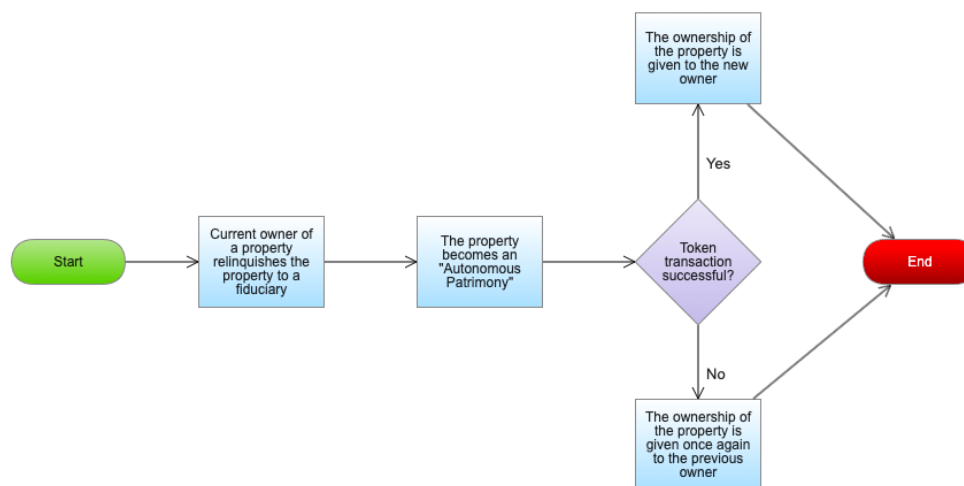


Figure 2: Flowchart for possible legal process

Design Challenges

First and foremost, there is the aspect of time, there is just around 2 months left and in this timeframe the author must also continue to learn aspects for understanding solidity and blockchain development as a whole. Therefore, a manageable scope and target must be defined for the project.

Apart from the previous, a first milestone must be reached in order to begin the designing process, this milestone is to find a way to deploy new smart contracts from command. This is the first step that will mark how the project can be implemented and therefore designed.

The first approach to solve this problem was with the use of a decentralized application (DApp) using a stablished architecture and boiler plate such as the one defined by Truffle React Box (Truffle-Box, 2017). After some meddling and experimenting with this possible path it became clear that DApps are thought out for interacting with already deployed smart contracts, but unable, at least in the explored methods for this project, to deploy new smart contracts.

Another path was explored in order to solve the problem of automating the process of deploying smart contracts, and this path was successful and is the basis from which the whole proof-of-concept of the project will be constructed. The solution found was to create an API with the Truffle framework that can run shell scripts in order to deploy a smart contract.

Project Scope and Target

As it was previously mentioned a reachable scope and target in terms of time had to be stablished for the implementation of the project. This scope must be enough to be an MVP and mark the first step to fulfill the necessities which were mentioned on the project's motivation. Taking these two building factors into account the following scope and target were defined:

Develop a web application that is able to create and deploy new smart contracts for a token standard. Any user, with the minimum requirement of understanding how to possess and manage a cryptocurrency wallet for storing tokens, must be able to consume the services from this application.

This scope is sufficiently minimal to be reachable within the timeframe of the project and it marks the first step to solve the necessities stablished on the project motivation. Having this application will enable users without technical knowledge to access the growing market of asset tokenization. And joining this, with the legal path for asset tokenization within Colombian Law, the necessities of opening this market to Latin America will be reached.

This proof of concept will only deploy one type of token, as it can then be replicated with any other typed of token. The token that was chosen for the purposes of this project was the ERC-20 token, as it is the most widely used, even with real estate asset tokenization, which is the focus of this project.

Functional Requirements

Name	R1 – Create new ERC-20 smart contract
Summary	Given the necessary inputs for creating an ERC-20 token, the application will create the smart contract for this token standard
Inputs	
Token name	
Token symbol	
Token amount	
Address of the wallet where the tokens will be stored	
Results	
A solidity file for an ERC-20 token that has the user's specifications	
A JavaScript file for deployment of the smart contract generated	

Name	R2 – Deploy a smart contract
Summary	Having a smart contract and its deployment file, the application must be able to deploy the smart contract on the Goerli test network of Ethereum
Inputs	
ERC-20 smart contract solidity file	
Deployment file	
Results	
A deployed token on the Goerli test network stored on the user's crypto wallet	

Application's Design and Flow

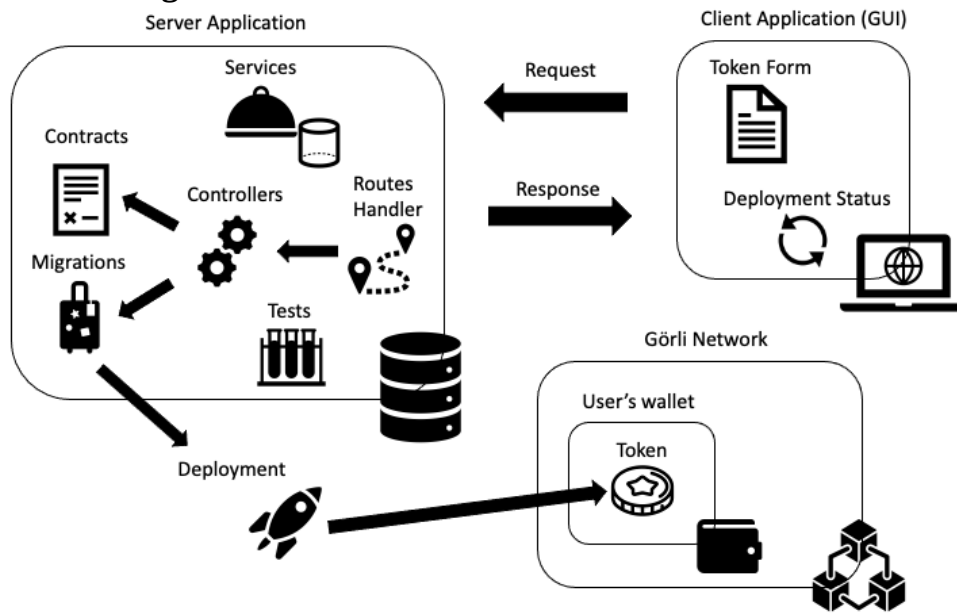


Figure 3: Diagram of the project's design and flow

On figure 3 a diagram of the project architecture is displayed. This project will follow a typical server-client web application where the client application will communicate with the server via an HTTP protocol sending requests to the server application and obtaining a response.

The client application will be a simple GUI with a form for creating a token and a section for checking the token's deployment status.

Although the application will only generate and deploy one token standard, a typical web server application architecture will be followed in order to make the application easily scalable. The server application will have a routes handler for managing the different token standard routes and directing them to the controller. The controller will handle the data from the request and create the contracts and the migrations (deployment files), once created, the controller will also have the task of deploying the smart contract to the Goerli test network and give the ownership of the tokens to the user's crypto wallet.

Additionally, the server will have a variety of services that can be consumed by any part of the server application and tests for the tokens.

Tools

- Visual Studio Code: A source-code editor made by Microsoft that will provide the environment for development.
- React: A JavaScript open-source library that is used to build user interfaces. The library will be used to implement the client application.
- NodeJS: A back-end JavaScript runtime environment. NodeJS will be used to implement the server application.
- Express: A backend web application framework for building RESTful APIs that will be used as the framework for building the API on the server.
- NPM (Node Package Manager): The default package manager for JavaScript runtime environment. It will be used in both applications to manage all libraries packages for installation.
- Uniqid: A node library that generates a unique hexadecimal identifier string based on the current time, process, and machine name. This library will be used to generate any needed ids within the platform.
- Truffle: A developing environment, testing framework and asset pipeline for Ethereum development. This environment will be used on the server applications to handle deployments among other Ethereum development tasks.
- OpenZeppelin | Contracts: A library for smart contract development that provides a variety of token standards implementation. It will be used to implement the ERC-20 Token smart contracts.

- **Faucets:** Faucets are a developer tool to get a testnet³ Ether to develop applications on a testnet. All blockchain transactions have a fee called gas that is the payment for the network validation. The Ethereum network runs on gas that is payed with Ether, while a testnet runs with another coin that does not have value and can be gained freely from a faucet.
- **Goerli-Network:** The testnet where the application will be developed. This testnet was mainly chosen because of the ability to find a reliable faucet. Testnets are used in development of blockchain applications in order to not spend real funds while developing.
- **Metamask:** Metamask is a cryptocurrency wallet that can interact with the Ethereum blockchain. It will be used as the cryptocurrency wallet that will both be the source of GoerliETH, the coin on the Goerli testnet, used for transactions, as for the wallet with different accounts where the tokens will be owned.
- **Ganache:** An application that can create a local Ethereum blockchain that can be used for development. As the faucets do not provide a great amount of test ETH, on the early stages of development, the application will run on this local Ethereum network instead of the testnet in order to save test funds
- **Web3:** A library used to interact with smart contracts and the blockchain network
- **HDWallet-Provider:** A library used to sign transactions for addresses derived from a 12- or 24-word mnemonic⁴. This library will be used to connect to the meta mask address that has the test ETH in order to sign these transactions.

³ "Testnets are a blockchain powered instance that can be used for experimenting and testing without the risk to spend real funds. [...] . Testnet coins distinct from an oficial (mainnet) coint" such as Ether "that do not have value and can be obtained freely from faucets." (Franco, 2014)

⁴ "A group of words that allow acc3ess to a cryptocurrency wallet" (Argent)

- Infura: “A Web3 backend and Infrastructure-as-a-Service (IaaS) provider that offers a range of services and tools for blockchain development” (Ivanotech, 2021). The main use of Infura in this project is as a blockchain node⁵ provider for the application.
- Mocha: A JavaScript testing framework that will be used to create the tests for the back-end services and tokens.
- Chai and Chai-As-Promised: Libraries that facilitate testing, and they will be used uniquely for testing within the server application.

Implementation

All of the code implemented for the project can be found on the following link to the public repository <https://tinyurl.com/pVallejoGradProject> (Vallejo Seade, 2022). The repository has all the iterations, versioning, issues, bugfixes among any other history of the project. However, in this section, only the final version of the implementation will be presented.

⁵ Blockchain nodes are the moderator that build the infrastructure of a decentralized network. Their primary function is to maintain consensus of a blockchain’s public ledger. [...] On the most basic level, a node is simply a device running the software of a specific blockchain (Becher, 2022)

Server Application Implementation

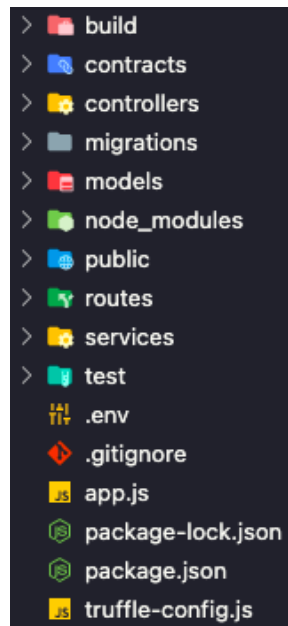


Figure 4: Folder and file structure of the server application

On figure 4 the folder and file structure of the server application is presented. In this section any important file will be explored in depth, and somewhat important files will be commented on. First, we begin with the `app.js` file, which marks the starting point of the server application, this file only creates a new server importing it from the file `models/server.js`, shown on figure 5, and then runs the `listen()` function to listen to any request being made to the server. The `models/server.js` creates a new server using express and it configures CORS⁶ (cross-origin resource sharing) routes middlewares and the `listen()` function.

⁶ "HTTP requests for resources from a different domain than the domain of the resource making the request" (HTTP access control - MDC, 2019)

```

const express = require('express');
const cors = require('cors');

class Server {

  constructor() {
    this.app = express();
    this.port = process.env.PORT;

    // Middlewares
    this.middlewares();

    // Application routes
    this.routes();
  }

  middlewares() {
    //CORS
    this.app.use(cors());

    // Body parser and reader
    this.app.use(express.json());

    // Public Directory
    this.app.use(express.static('public'));
  }

  routes() {
    this.app.use('/api/erc20', require('../routes/erc20'));
  }

  listen() {
    this.app.listen(this.port, () => {
      console.log('Server running on port', this.port);
    });
  }
}

```

Figure 4: *models/server.js* file

Next comes the *routes/erc20.js* handles the different routes for the ERC-20 services provided by the API. Only two routes are in use, the route that handles the GET requests with an id, that provide the user with the information of a token created in the platform, and the POST request that allows the user to create an ERC-20 smart contract and deploy it. The file that handles the creation of all the token associated files and the deployment of the file is the *controllers/erc20.js* file. This file makes use of the two currently available services in the application:

1. The console events handler service: Provided by the *services/console-events.js* file, it is a service that defines the structure of messages shown in the console of the server. This service is implemented to standardize and facilitate the understanding of the flow within the application and debugging.
2. The database service: Provided by the *services/database.js* file, within the scope of this project, the need of having an actual database was not needed.

Therefore, to simplify and save time, a simulated database within the application is created that consists of an array of json objects. This service handles any type of interaction with the simulated database.

The *controllers/erc20.js* contain two main controller functions called by the routes:

1. `erc20Get(req, res)`: Takes the request parameter of the token id, then looks in the database for a token that matches the input id and if found it attaches the json object of the Token in the response with a response code 200⁷, if not found, the response sends a json file with a message indicating that the resource could not be found and a 404⁸ response code.
2. `erc20Post(req, res)`: Displayed on figure 5, checks if there are no tokens with the same name or symbol, although this is allowed within the Ethereum blockchain, it is not allowed on truffle. If there is already a token with the name or symbol indicated by the user on the POST request, the function will send a response message indicating so with a 406⁹ response code. If there are no tokens with the same token name or symbol, the begins the chain of processes in order to register the token in the application and then deploying it on the testnet, in this case the function sends a response with the object created on the DB and a 201¹⁰ response code.

⁷ Ok response code that indicates that the result from the request was successful

⁸ Not found response code indicating that the resource being searched could not be found within the server

⁹ Not acceptable response code indicating that the request is not acceptable for some reason.

¹⁰ Created response code indicated that a resource was created successfully within the application

```

/**
 * Controller for the post request of an ERC-20 token
 * @param {*} req Request to the server
 * @param {*} res The response from the server
 */
const erc20Post = (req = request, res = response) => {

  // Request body
  const { name, symbol, tokenAmount, ownerAddress } = req.body;

  console.log(req.body)

  const tokenObj = {
    id: `${uniqid()}`,
    standard : 'ERC-20',
    name : name,
    symbol : symbol,
    amount : tokenAmount,
    contractAddress : '0' // The contract address is 0 when the contract has not been deployed
  }

  if(pushElementInDatabase(tokenObj)){
    oneLineConsoleMessage(ELEMENT_ADDED_TO_DB, SUCCESS, `The token ${name} has been succesfully added to the DB`);

    // Templates needed for deployment and contract creation
    const deploymentFile = deploymentFileTemplate(name);
    const contractFile = erc20ContractTemplate(name, symbol, tokenAmount, ownerAddress);

    // Creation of the needed files and deployment of contract
    createERC20FilesAndDeployContract(tokenObj, deploymentFile, contractFile);

    res.status(201).json(tokenObj);
  } else {
    oneLineConsoleMessage(ELEMENT_ADDED_TO_DB, ERROR, `The token ${name} could not be added to the DB because there is `+
    `already a token with the name ${name} or the symbol ${symbol}`);

    res.status(406).json({
      msg : `There is already a token with the name ${name}`,
      key : 'DUPLICATE_TOKEN'
    });
  }
}

```

Figure 5: erc20Post (req, res) function available in the *controllers/erc20.js* file

The streps taken by erc20Post (req, res) function initiate the process of deployment is to first register the token object within the database using the function imported from the database service pushElementInDatabase (newElement) which receives the object through the parameters and then proceeds to store the object within the database as long as there are not already any tokens stored with the same name or symbol, in this case, the function returns a true value, on the contrary, the function returns false and does not store the object in the database. The object stored within the database has the following attributes:

- id: The identifier of the token instance within the database, this identifier is generated via the function uniqid() from the uniqid library that generated a unique identifier based on the current time, process and machine name.
- standard: The token standard of the token being generated.

- name: The name of the token being generated.
- symbol: The symbol of the token being generated.
- amount: The initial number of tokens that want to be created
- contractAddress: The address where the generated token smart contract resides.

It is initiated with '0' as a value indicating that the token is not deployed yet and therefore has no address.

After a successful insertion to the database, the function proceeds to use the helper functions `deploymentFileTemplate(name)` and `erc20ContractTemplate(name, symbol, tokenAmount, ownerAddress)`, both taking in their parameters the data to create the token and returning a string with the templates for the deployment file and smart contract file respectively. Once this strings are generated, the function proceeds to call the `createERC20FilesAndDeployContract(tokenObj, deploymentFile, contractFile)` helper function, this function can be seen on figure 6.

```
/**
 * Creates the deployment and contract files for an ERC-20 token and also deploys the contract created
 * @param {*} tokenObj The object of the token being created on the format stored in the DB
 * @param {*} deploymentFile The content of the deployment file
 * @param {*} contractFile The content of the contract file
 */
function createERC20FilesAndDeployContract(tokenObj, deploymentFile, contractFile) {
  fs.writeFile(path.join(__dirname, '..', 'migrations', '1_deploy_contracts.js'), deploymentFile, (err) => {
    if (err) {
      multiLineConsoleMessage(FILE_UPDATE,
        ERROR,
        `The solidity file for the token ${tokenObj.name} could not be written due to the error shown below`,
        `${err.message}`);

      if(deleteTokenById(tokenObj.id)){
        oneLineConsoleMessage(ELEMENT_DELETED_FROM_DB, SUCCESS, `The token ${tokenObj.name} ` +
          `has succesfully been deleted from the database due to an error writing the deployment file for the token`);
      } else {
        oneLineConsoleMessage(ELEMENT_DELETED_FROM_DB, ERROR, `The token ${tokenObj.name} could not be deleted from the DB ` +
          `when an error writting the deployment file for the token occurred`);
      }

      return ;
    }
    oneLineConsoleMessage(FILE_UPDATE, SUCCESS, `The deployment file in order to deploy the token ${tokenObj.name} ` +
      `was succesfully updated`);

    createContractFile(tokenObj, contractFile);
  });
}
```

Figure 6: `createERC20FilesAndDeployContract(tokenObj, deploymentFile, contractFile)` function available in the `controllers/erc20.js` file

This function writes the deployment file inside the *migrations* folder, if some error occurs trying to write this file, the application indicates there was an error and prints it and if

the file was written successfully, the application also indicates the success and proceeds to call the `createContractFile(tokenObj, contractFile)` function shown on figure 7.

```
/**
 * Creates the contract file for the Token
 * @param {*} tokenObj The object of the token being created on the format stored in the DB
 * @param {*} contractFile Content of the contract file
 * @return True if the operation was completed succesfully, false otherwise
 */
function createContractFile(tokenObj, contractFile){
  fs.writeFile(path.join(__dirname, '..', 'contracts', `${tokenObj.name}.sol`), contractFile, (err) => {
    if (err) {
      multilineConsoleMessage(FILE_CREATION,
        ERROR,
        `The solidity file for the token ${tokenObj.name} could not be created due to the error shown below`,
        `${err.message}`);

      if(deleteTokenById(tokenObj.id)){
        oneLineConsoleMessage(ELEMENT_DELETED_FROM_DB, SUCCESS, `The token ${tokenObj.name} ` +
          `has succesfully been deleted from the database due to an error writing the contract file for the token`);
      } else {
        oneLineConsoleMessage(ELEMENT_DELETED_FROM_DB, ERROR, `The token ${tokenObj.name} could not be deleted from the DB ` +
          `when an error writting the contract file for the token occurred`);
      }

      return;
    }

    oneLineConsoleMessage(FILE_CREATION, SUCCESS, `The solidity file for the token ${tokenObj.name} was succesfully written`);

    deployContract(tokenObj);
  });
}
```

Figure 7: `createContractFile(tokenObj, contractFile)` function available in the `controllers/erc20.js` file

The `createContractFile(tokenObj, contractFile)` function, similarly to the previous function explored, creates the solidity file for the token contract, if any errors occur on this process the user is informed and the token object deleted from the database, and if successful, the function proceeds to call the `deploy(tokenObj)` function.

```

/**
 * Runs the script in order to deploy contracts
 * @param {*} tokenObj The object of the token being created on the format stored in the DB
 * @return True if the contract was deployed correctly on the network, false otherwise
 */
function deployContract(tokenObj) {
  exec("truffle migrate --network goerli_infura", (error, stdout, stderr) => {
    if (error) {
      multiLineConsoleMessage(TRUFFLE_MIGRATION,
        ERROR,
        `The error shown below occurred while trying to run 'truffle migrate' command for deploying ` +
        `the token with the name: ${tokenObj.name}`,
        `${error}`);

      if(deleteTokenById(tokenObj.id)){
        oneLineConsoleMessage(ELEMENT_DELETED_FROM_DB, SUCCESS, `The token ${tokenObj.name} ` +
        `has succesfully been deleted from the database due to an error running 'truffle migrate' command`);
      } else {
        oneLineConsoleMessage(ELEMENT_DELETED_FROM_DB, ERROR, `The token ${tokenObj.name} could not be deleted from the DB ` +
        `when an error running 'truffle migrate' command occurred`);
      }
    }

    return;
  });

  if (stderr) {
    multiLineConsoleMessage(TRUFFLE_MIGRATION,
      SUCCESS,
      `The command 'truffle migrations was succesfully runned for deploying the token with the name: ` +
      `${tokenObj.name} and obtained the shell stream shown below`,
      `${stderr}`);

    if(deployContractOfAnElementById(tokenObj.id, tokenObj.name)){
      oneLineConsoleMessage(ELEMENT_UPDATED_FROM_DB, SUCCESS, `The token ${tokenObj.name} ` +
      `has succesfully updated its contractAddress property due to a succesfull deployment of the contract`);
    } else {
      oneLineConsoleMessage(ELEMENT_UPDATED_FROM_DB, ERROR, `The token ${tokenObj.name} ` +
      `could not update its contractAddress property due to a succesfull deployment of the contract`);
    }
  }

  return;
}

multiLineConsoleMessage(TRUFFLE_MIGRATION,
  SUCCESS,
  `The command 'truffle migrations was succesfully runned for deploying the token with the name: ` +
  `${tokenObj.name} and obtained the shell output shown below`,
  `${stdout}`);

if(deployContractOfAnElementById(tokenObj.id, tokenObj.name)){
  oneLineConsoleMessage(ELEMENT_UPDATED_FROM_DB, SUCCESS, `The token ${tokenObj.name} ` +
  `has succesfully updated its contractAddress property due to a succesfull deployment of the contract`);
} else {
  oneLineConsoleMessage(ELEMENT_UPDATED_FROM_DB, ERROR, `The token ${tokenObj.name} ` +
  `could not update its contractAddress property due to a succesfull deployment of the contract`);
}
});
}

```

Figure 8: deployContract(tokenObj) function available in the controllers/erc20.js file

The `deploy(tokenObj)` function shown on figure 8 runs the script in order to deploy the tokens on the Goerli testnet. Once again, the applications inform an eventual success or failure of this deployment and the relevant information associated here. Once the token is successfully deployed, the application changes the instance of the token within the database to register the address where the token resides within the Goerli test network. On figure 9 the server's console after a successful deployment is shown.

```

ELEMENT-ADDED-TO-DB/SUCCESS[2022-12-06T18:19:54.858Z]: The token MyToken has been succesfully added to the DB
FILE-UPDATE/SUCCESS[2022-12-06T18:19:54.862Z]: The deployment file in order to deploy the token MyToken was succesfully updated
FILE-CREATION/SUCCESS[2022-12-06T18:19:54.866Z]: The solidity file for the token MyToken was succesfully written

TRUFFLE-MIGRATION/SUCCESS:[2022-12-06T18:20:51.757Z]: The command 'truffle migrations was succesfully runned for deploying the token with t
he name: MyToken and obtained the shell stream shown below'
-----
beginning
- Blocks: 0          Seconds: 0
- Blocks: 0          Seconds: 4
- Blocks: 1          Seconds: 8
-----
ending
ELEMENT-UPDATED-FROM-DB/SUCCESS[2022-12-06T18:20:51.766Z]: The token MyToken has succesfully updated its contractAddress property due to a su
ccesfull deployment of the contract

```

Figure 9: Sever's console of a successful token deployment

When a token is successfully deployed, a series of files are generated within the `build/contracts` folder: `Context.json`, `ERC20.json`, `IERC20.json`, `IERC20Metadata.json`, `<TokenName>.json`. All these files are automatically created by Truffle and OpenZeppelin | Contracts, and these are needed by Truffle in order to deploy the smart contract.

Now, the `truffle-config.js` file will be explored, as its name implies, this file's purpose is to specify the configurations that will be used for the Truffle environment. This file exports an object with all of the truffle configurations, most of this configurations were left on its default values with the exception of the networks configuration that is shown on figure 10.

```

networks: {
  // Useful for testing. The `development` name is special - truffle uses it by default
  // if it's defined here and no other network is specified at the command line.
  // You should run a client (like ganache, geth, or parity) in a separate terminal
  // tab if you use this network and you must also set the `host`, `port` and `network_id`
  // options below to some value.
  //
  development: {
    host: "127.0.0.1",    // Localhost (default: none)
    port: 7545,          // Standard Ethereum port (default: none)
    network_id: "*",     // Any network (default: none)
  },
  ganache_local:{
    provider: function(){
      return new HDWalletProvider(MNEMONIC, 'http://127.0.0.1:7545', AccountIndex)
    },
    network_id: 5777
  },
  goerli_infura:{
    provider: function(){
      return new HDWalletProvider(MNEMONIC, 'https://goerli.infura.io/v3/${INFURA_API_KEY}', AccountIndex)
    },
    network_id: 5
  },
}

```

Figure 10: Project truffle configuration available on the `truffle-config.js` file

The configured networks mark the blockchain network that Truffle will deploy the contracts. The first network is the default network provided by Truffle, which is a local network

that does not simulate very well a real blockchain network and it is very limited. The second network configured is the ganache network, which is an improvement from the Truffle development network and simulates a newly created blockchain network much better and provide better tools; in previous iterations of development this was the network used in order to save GoerliETH which is scarce and hard to access. The last network is the goerli network which is the one being used for the iteration presented on this document.

On figure 10, it is noticeable the use of HDWalletProvider for both the ganache and Goerli network, it is used to configure the signature of transactions of all the transactions being made in Truffle to a specified wallet. In the case of this project, this wallet is a Metamask wallet that contains all the GoerliETH. In order to configure the wallet with HDWalletProvider, the wallet's mnemonic needs to be provided, the value of this mnemonic is stored in the dotenv file as it is private and sensitive data.

```
contract("Token Test", async (accounts) => {
  const [deployerAccount, receipient, anotherAccount] = accounts;

  it("is not possible to send more tokens than available in total", async() =>{
    let instance = await MyToken.deployed();
    let balanceOfDeployer = await instance.balanceOf(deployerAccount);

    expect(instance.transfer(receipient, new BN(balanceOfDeployer+1))).to.eventually.be.rejected;
    expect(instance.balanceOf(deployerAccount)).to.eventually.be.a.bignumber.equal(balanceOfDeployer);
  })

  it("is possible to send tokens between accounts", async() => {
    const sendTokens = 1;
    let instance = await MyToken.deployed();
    let totalSupply = await instance.totalSupply();

    expect(instance.balanceOf(deployerAccount)).to.eventually.be.a.bignumber.equal(totalSupply);
    expect(instance.transfer(receipient, sendTokens)).to.eventually.be.fulfilled;
    expect(instance.balanceOf(deployerAccount)).to.eventually.be.a.bignumber.equal(totalSupply.sub(new BN(sendTokens)));
    expect(instance.balanceOf(deployerAccount)).to.eventually.be.a.bignumber.equal(new BN(sendTokens));
  })
})
```

Figure 11: Tests available on the *test/ERC20Token.test.js* file

On figure 11 the simple tests to verify the proper behavior of tokens are written, the test verify that it is not possible to send more tokens than the ones created and that it is possible to transfer tokens within accounts. The purpose of this tests (although not extensive and in this case even redundant as the token smart contracts being used are provided by OpenZeppelin who already test their smart contracts) is to build a testing foundation for the application when

tokens such as ERC-4907 (the rental token standard) that are not implemented by OpenZeppelin can be tested. On figure 12, the results from running the latter tests are displayed.

```
Compiling your contracts...
=====
> Everything is up to date, there is nothing to compile.

Contract: Token Test
  ✓ is not possible to send more tokens than available in total (83ms)
  ✓ is possible to send tokens between accounts (3891ms)

1) "after each" hook: after test in "Contract: Token Test"

2 passing (5s)
```

Figure 12: Results from tests implemented on the *test/ERC20Token.test.js* file

Client Application Implementation

It is not worth getting into the details of the code for the client application. This application is a simple single-page application developed with ReactJS that consumes the services provided by the server application getting the input from the user from a form. The GUI that comes from the result of this application is shown on figure 13.

The screenshot shows a web application titled "Asset Tokenization Tool". It features a dark-themed interface with a form for creating tokens. The form has six input fields arranged in two columns. The left column contains "Asset Type" (a dropdown menu with "Real-State" selected), "Token Standard" (a dropdown menu with "ERC-20" selected), and "Token Name" (a text input field with placeholder text "Write the token's name..."). The right column contains "Token Symbol" (a text input field with placeholder text "Write the token's symbol..."), "Token Amount" (a text input field with placeholder text "Write the initial token amount..."), and "Owner's Address" (a text input field with placeholder text "Write the address of the token owner..."). A blue "TOKENIZE" button is located at the bottom right of the form. At the bottom of the page, there is a footer with the text: "Universidad de Los Andes - Facultad de Ingeniería - Departamento de Ingeniería de Sistemas y Computación - Pedro Vallejo (201825720)".

Figure 13: Application GUI

As it was mentioned before, the GUI is conformed mainly of a form that contains the elements which will be used to create the token. When all the fields of the form are filled and

the TOKENIZE button pressed, the application wraps all this data into a json object and then creates an http request to the erc20Post route within the server application. When the button is pressed, the GUI shows another component that can be seen on figure 14.

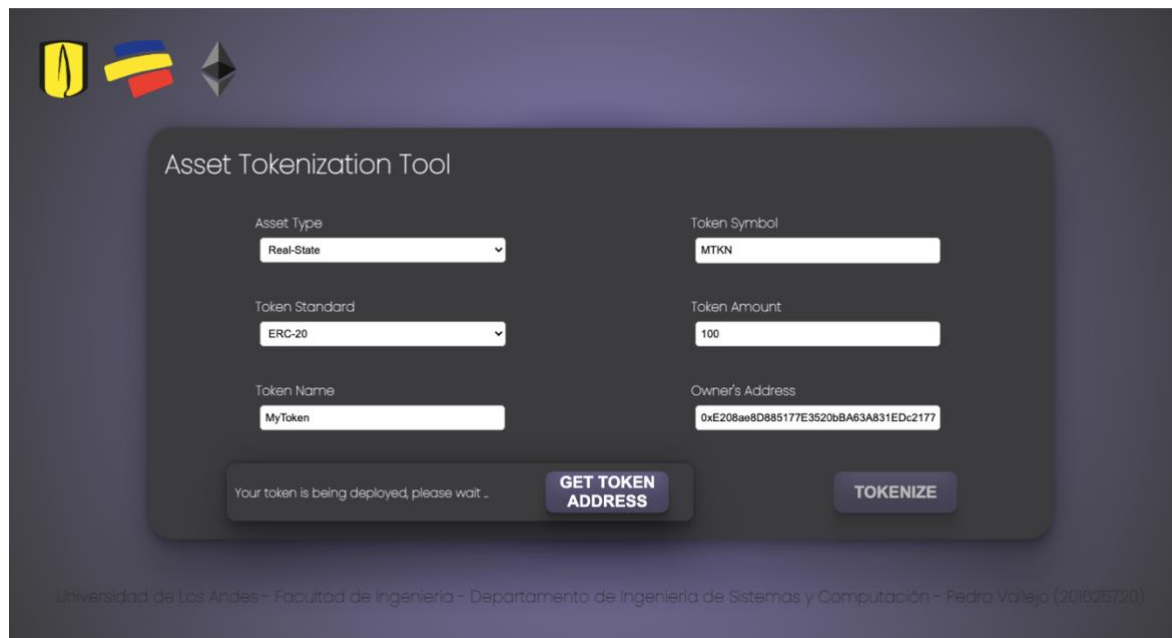


Figure 14: Application GUI after the TOKENIZE button is pressed with all the data filled

This new component visible on the bottom left part of the card holding the form indicates the user that the token is being deployed, and the user can click on the GET TOKEN ADDRESS button to see if the token has been deployed yet, if it still not deployed the component will remain the same. This button basically makes a call to the erc20Get route on the server passing through the request params the id of the token, which is stored on the client application when the post request was executed and received the object representing the token on the database. The component will change when the object obtained by the call has an attribute of contractAddress different to '0'. When this eventually happens, the component changes as shown on figure 15.

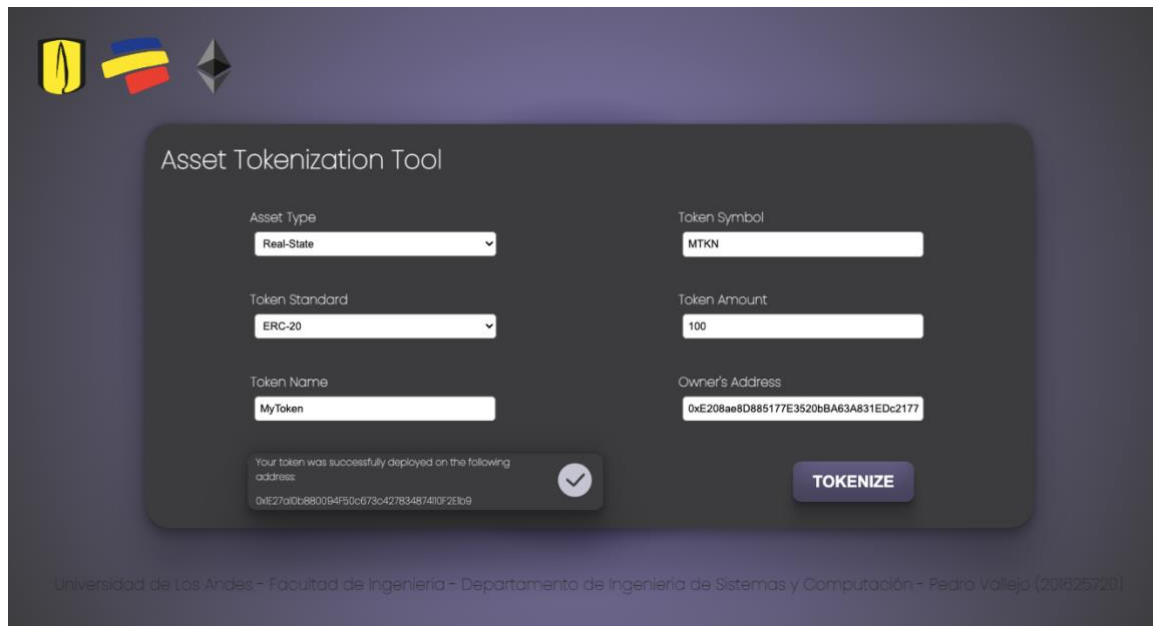


Figure 15: Application GUI after the token was deployed

The component now changes to indicate that the token has been successfully deployed and provides the user the address where the token resides within the blockchain network. In Metamask, this address is needed to import a token and effectively see the quantity of said token owned by the wallet. Once the token is imported this information can be seen, as it shown on figure 16.

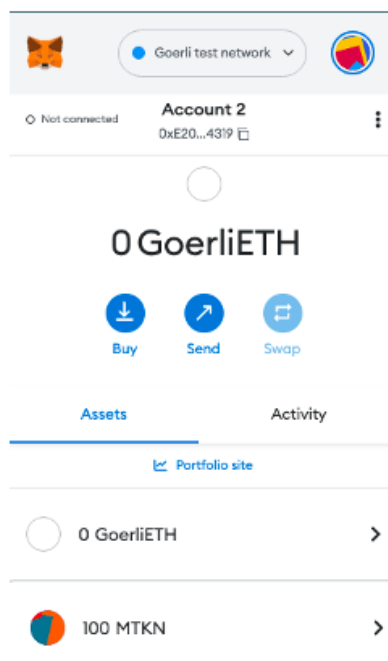


Figure 16: Metamask displaying ownership of the token deployed by the application

The transaction details for the creation and deployment of a token can be seen using Etherscan, a platform for accessing all the transactions that are being made within the Goerli test network and their details. The details of a token created by the project's application is shown on figure 17.

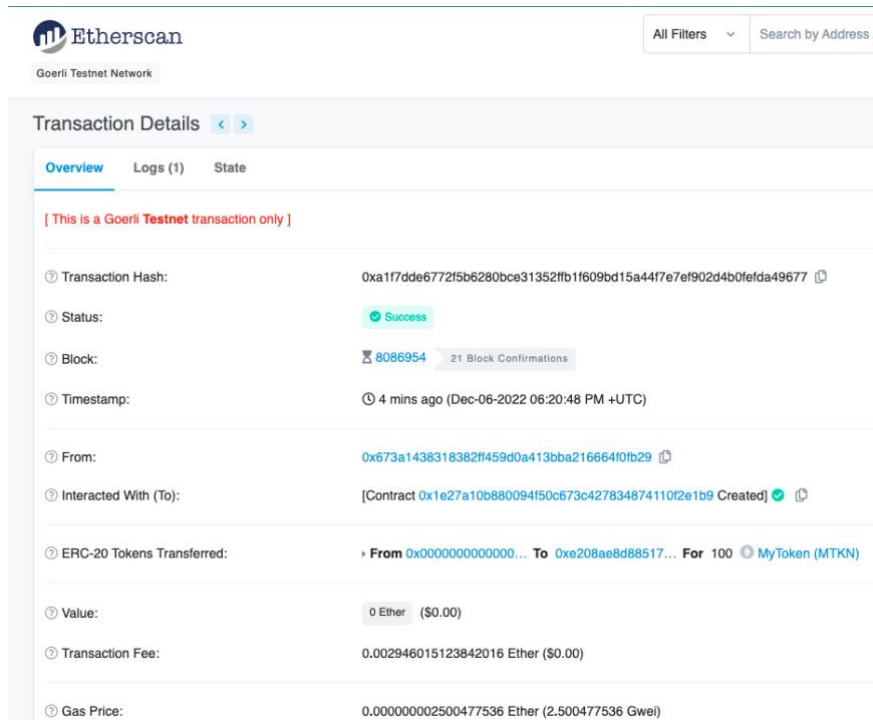


Figure 17: Transaction details of a token generated by the application

Results

Of course, the application not always works perfectly, there are problems sometimes deploying a token as the one shown on figure 18. For this reason and for understanding better the outcomes given by the application this section is included. In this section through the behavior of the application will be observed.

```
TRUFFLE-MIGRATION/ERROR:[2022-12-06T18:18:47.717Z]: The error shown below occurred while trying to run 'truffle migrate' command for deployin
g the token with the name: MyToken
----- beginning
Error: Command failed: truffle migrate --network goerli_infura
/Users/pedrovallejoseade/Desktop/GraduationProject-pvallejo/server/node_modules/eth-block-tracker/src/polling.js:51
  const newErr = new Error('PollingBlockTracker - encountered an error while attempting to update latest block:\n${err.stack}')
                  ^
Error: PollingBlockTracker - encountered an error while attempting to update latest block:
undefined
    at PollingBlockTracker._performSync (/Users/pedrovallejoseade/Desktop/GraduationProject-pvallejo/server/node_modules/eth-block-tracker/s
rc/polling.js:51:24)
    at processTicksAndRejections (node:internal/process/task_queues:96:5)
    at runNextTicks (node:internal/process/task_queues:65:3)
    at listOnTimeout (node:internal/timers:528:9)
    at processTimers (node:internal/timers:502:7)
----- ending
```

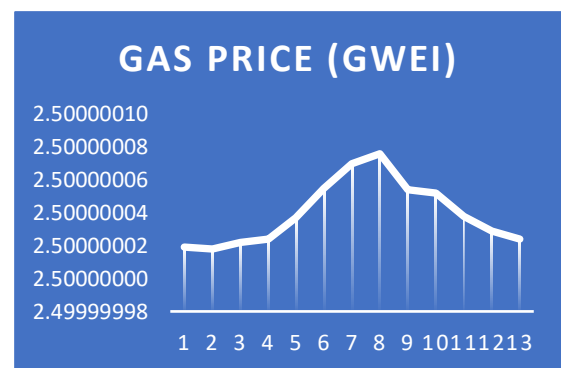
Figure 18: Transaction details of a token generated by the application

The following table registers the outcome of deployment of multiple tokens in the application. These tokens were deployed on a similar time interval.

Token number	Deployment time (ms)	Transaction fee (Ether)	Gas price (Gwei)	Error/Exception
1	54077	0.002945482522385667	2.500000019	-
2	56235	0.002945482521207474	2.500000018	-
3	51866	0.002945452525919982	2.500000022	-
4	40332	0.002945482528276632	2.500000024	-
5	50280	0.002945452543592697	2.500000037	-
6	51569	0.002945482564800615	2.500000055	-
7	69419	0.002945482582473510	2.500000070	-
8	-	-	-	PollingBlockTracker
9	55315	0.002945482589542668	2.500000076	-
10	47902	0.002945482563622422	2.500000054	-
11	54534	0.002945482561266036	2.500000052	-
12	57048	0.002945482544771334	2.500000038	-
13	51787	0.002945482534167597	2.500000029	-
14	-	-	-	PollingBlockTracker
15	58953	0.002945482528276632	2.500000024	-

AVG	53793.62	0.002945477931561790	2.50000003985
-----	----------	----------------------	---------------

Success Rate	86.7%
--------------	-------



Further work

The target of this project was to mark the first step into easy access to asset tokenization, and the first step is to facilitate the creation of a token to any user with enough knowledge to manage a cryptocurrency wallet such as Metamask. This target is met with the implementation of the project at the moment, but this section is meant to recommend the possible next steps.

Further work on The Server Application

The next logical step for the server-side application is to handle the Polling Block Tracking errors adequately since they seem to be the most common error while trying to deploy a token's smart contract. Once this is done, implementing new token standards that are relevant for real estate asset tokenization. In order to do this a database must be implemented to store and handle all of the different token standards. Token standards that are of interest of the real estate industry are the following: ERC-721, ERC-777, ERC-1155 and the ERC-4907. The platform is scalable in this sense so, for this token standards the actual implementation of the ERC-20 can be replicated once the adaptation of a real database is done.

Also, if the current platform would be used in actual production it must find a way of financing, the most obvious one would be to charge the user for the service in order to afford the transaction fees, gas prices and the possible deployment servers on the cloud.

Further work on The Client Application

The client application requires the most work, due to time restraints this application was not build with a scalable structure, it was built to serve the sole purpose of providing a GUI for the user to consume the current services available on the server application. Therefore, next step would be to restructure completely this application in order to allow more token standards.

Future platform

The project at the moment only has the capacity to be this first step to facilitate asset tokenization to a wider public, therefore it must serve as a basis for a future platform which can satisfy the whole process that involves this necessity. The next platform must not only

allow to generate tokens but also to interact with them via web3. The latter is only in terms of functionality, the most important aspect for a future platform will be on the client application.

This application must not only be very user friendly but should be able to provide the user with the information needed to fully (or at least as much as possible) understand the implications of asset tokenization and the advantages and disadvantages of the different standards based on the type of asset being tokenized.

Conclusions

The target set up for this project were met, certainly a more experienced blockchain developer could implement a better version of what was implemented for this project. But the importance behind this project, as with any project in engineering, is not to obtain perfection on the first try but to construct something and then iterate over this repeatedly making it each time better. The journey for this project started from having an idea of what blockchain is, to fully researching a real-world use case, finding a necessity and learning blockchain development while almost at the same time constructing a solution using this technology, many times facing problems with no documented solutions and having to ideate an own solution joining bits and pieces of information.

If nothing else, this project is a documentation of the engineering process with all the stages of looking into different paths constantly until a way ahead is found.

Nonetheless I truly believe that this project and the documentation of the project can mark the beginning of an iterative process that can eventually fully, or at least to a great degree, solve the issues that were stated.

References

Argent. (s.f.). *What is a mnemonic seed phrase / backup phrase*. Retrieved from Argent.xyz: <https://www.argent.xyz/what-is-a-mnemonic-seed-phrase-crypto/#::~:~:text=A%20mnemonic%20phrase%2C%20mnemonic%20seed,this%20isn't%20quite%20correct.>

- Becher, B. (29 de September de 2022). *What Are Blockchain Nodes and How Do They Work?* Obtenido de BuiltIn: <https://builtin.com/blockchain/blockchain-node>
- Cryptopedia Staff. (28 de June de 2022). *Digital Assets: Cryptocurrencies vs. Tokens*. Retrieved from Gemini: <https://www.gemini.com/cryptopedia/cryptocurrencies-vs-tokens-difference>
- Davis, J. (3 de October de 2011). *The New Yorker*. Obtenido de The Crypto-Currency: Bitcoin and its mysterious inventor: <https://www.newyorker.com/magazine/2011/10/10/the-crypto-currency>
- Franco, P. (2014). *Understanding Bitcoin: Cryptography, Engineering and Economics*. New York: John Wiley & Sons.
- Gerencie. (25 de November de 2021). *Contrato de fiducia mercantil*. Retrieved from Gerencie.com: <https://www.gerencie.com/fiducia-mercantil.html>
- Gupta, A., Rathod, J., Patel, D., Bothra, J., Shanbhag, S., & Bhalero, T. (2020). Tokenization of Real Estate Using Blockchain Technology. *Lecture Notes in Computer Science (LNSC, volume 12418)*, 77-99.
- Hancock, M., & Vaizey, E. (2016). *Distributed Ledger Technology: beyond block chain*. London: Government Office for Science (UK).
- Houben, R., & Snyers, A. (2018). *Cryptocurrencies and blockchain*. Strasbourg: European Parliament.
- HTTP access control - MDC. (27 de May de 2019). Retrieved from Developer.mozilla.org: https://web.archive.org/web/20100527153021/https://developer.mozilla.org/En/HTTP_access_control
- Ivanotech. (16 de June de 2021). *Infura Explained - What is Infura*. Retrieved from Academy Moralis: <https://academy.moralis.io/blog/infura-explained-what-is-infura>
- Keynes, J. M. (1930). Liquid Capital. En J. M. Keynes, *A Treatise on Money* (pág. 116). Cambridge: Cambridge University Press.
- Kumar, S., Suresh, R., Liu, D., Kronfellner, B., & Kaul, A. (2022). *Relevance of on-chain asset tokenization in 'crypto winter'*. Boston: Boston Consulting Group.
- Lachar, N., Elkhail, A. A., Bacha, A., & Malik, H. (2020). *A Cross-Stack Approach Towards Defending Against Cryptojacking*. Piscataway, New Jersey: IEEE.
- Maldonado, J. (15 de September de 2022). *Las principales compañías que venden inmuebles tokenizados en blockchain*. Retrieved from Observatorio Blockchain: <https://observatorioblockchain.com/tokenizacion/las-principales-companias-que-venden-inmuebles-tokenizados-en-blockchain/>
- Market Business News. (2021). *What is fungible?* Retrieved from Market Business News: <https://marketbusinessnews.com/financial-glossary/fungible-definition-meaning/>
- Narayanan, A., Bonneau, J., Felten, E., Miller, A., & Goldfeder, S. (2016). *Bitcoin and cryptocurrency technologies: a comprehensive introduction*. New Jersey : Princeton University Press.
- Osorio, S. (2022). *Introducción a tokens ethereum*. Retrieved from Platzi: <https://platzi.com/clases/2991-introduccion-tokens-ethereum/48568-que-es-un-token/>

- RealT. (2021). *Who holds the deed to the house?* Retrieved from RealT Wiki:
<https://wiki.realt.co/far/who-holds-the-deed-to-the-house>
- Szabo, N. (1994). *Smart Contracts*. retrieved from Phonetic Sciences, Amsterdam:
<https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwin terschool2006/szabo.best.vwh.net/smart.contracts.html>
- Tapscott, D., & Tapscott, A. (2016). *Blockchain Revolution: How the Technology Behind Bitcoin is Changing Money, Business, and the World*. New York: Penguin Random House LLC.
- Tian, Y., Adriends, P., Minchin, R. E., Chang, C., Lu, Z., & Qi, C. (3 de May de 2021). *Asset Tokenization: A blockchain Solution to Financing Infrastructure in Emerging Markets and Developing Economies*. Obtenido de SSRN Papers:
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3837703
- Truffle-Box. (5 de February de 2017). *React-Box*. Retrieved from GitHub: <https://github.com/truffle-box/react-box>
- Usman, C. W. (6 de January de 2021). *SSRN Papers*. Retrieved from The Double Spending Problem and Cryptocurrencies: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3090174
- Vallejo Seade, P. (14 de November de 2022). *Graduation Project pvallejo Repository*. Retrieved from GitHub: <https://github.com/UnfriendlyKoala/GraduationProject-pvallejo>
- Vance, A. (14 de November de 2013). *Bloomberg*. Retrieved from Bitcoin Mining Chips, a High-Tech Arms Race: Faster processors will give digital-currency miners an edge:
<https://www.bloomberg.com/news/articles/2013-11-14/2014-outlook-bitcoin-mining-chips-a-high-tech-arms-race>
- Yaga, D., Mell, P., Roby, N., & Scarfone, K. (2018). Blockchain Technology Overview. *NISTIR 8202*, 32-33.
- Yip, J. (7 de June de 2022). *A New Fractional Way To Own Artworks*. Retrieved from Luxuo:
<https://www.luxuo.com/culture/art/a-new-fractional-way-to-own-artworks.html>